

PROJECT REPORT

Object Oriented Programming using C++

1. Project Title

Warehouse Inventory Management System (Project Code: CPP25)

2. Student Details

Item	Details
Name	Krishnraj Jadeja
Enrollment Number	IU2541230455
Class / Section	CSE, Section E
Project Code	CPP25
Language / Tools	C++ (g++), GitHub Codespaces, PlantUML
GitHub Repository	https://github.com/krishnrajjadeja/IU2541230455_Warehouse

3. Problem Statement

Small and medium warehouses often track products, suppliers, stock quantities and orders on paper or in scattered spreadsheets. This leads to wrong stock counts, products running out unnoticed, duplicate records and lost data. A simple, reliable program is needed that keeps product, supplier, stock and order information in one place, updates stock automatically whenever goods arrive or leave, warns about low stock, and remembers all data between runs.

4. Objectives

- To design the system using object oriented principles with separate, well-defined classes.
- To manage products, suppliers, stock and orders with add, display, search, update and delete operations.
- To automatically increase stock on incoming orders and decrease stock on outgoing orders.
- To prevent invalid data using validation and exception handling.
- To store data permanently in text files so that it is available after the program is reopened.
- To use dynamic memory allocation correctly and release it without leaks.
- To document the design with UML diagrams (Use Case, Class and Sequence).

5. Features

The program is a console, menu-driven application. The main menu offers the following operations:

No.	Menu option	Description
1	Add Record	Adds a new product (with opening stock and reorder level) or a new supplier. Duplicate IDs are rejected.
2	Display Records	Lists all products with their stock, or all suppliers.
3	Search Record	Finds a product or supplier by its ID.
4	Update Record	Changes the name, category and price of a product, or the name, phone and city of a supplier.
5	Delete Record	Removes a product (and its stock) or a supplier.
6	Main Transaction	Places an incoming or outgoing order and updates stock; also shows all orders.
7	Report	Shows counts, total stock value and the list of low stock items.
8	Save / Load	Saves all data to files or loads it back.
9	Exit	Saves all data automatically and closes the program.

Additional features: input validation (negative price, wrong phone number, non-positive quantity), a low stock warning when quantity falls below the reorder level, and automatic loading of saved data at program start.

6. Classes and Responsibilities

The system uses nine classes. Item and Order are abstract base classes; Product, Supplier, IncomingOrder and OutgoingOrder are derived classes. Warehouse is the system class that holds all entity objects and implements the business logic.

Class	Responsibility	Data members and important member functions
Item (abstract)	Common base for Product and Supplier.	Data: id, name (protected). Functions: getId(), getName(), setName(), pure virtual display() and toFileString().
Product	Represents an item stored in the warehouse.	Data: category, price (private). Functions: getters/setters, setPrice() with validation, display(), toFileString().
Supplier	Represents a supplier of goods.	Data: phone, city (private). Functions: setPhone() validates 10 digits, display(), toFileString().
Stock	Holds the quantity of one product.	Data: productId, quantity, reorderLevel (private). Functions: addStock(), removeStock() (throws if insufficient), isLow(), display().

Class	Responsibility	Data members and important member functions
Order (abstract)	Common base for all orders.	Data: orderId, productId, quantity, date (protected). Functions: pure virtual process() and getType(), display(), toFileString().
IncomingOrder	Order received from a supplier.	process() calls addStock(); getType() returns INCOMING.
OutgoingOrder	Order sent to a customer.	process() calls removeStock(); getType() returns OUTGOING.
Warehouse	System class: stores all objects and performs all operations.	Data: dynamic arrays products, suppliers, stocks, orders with counts and capacities; a FileManager object. Functions: add/display/search/update/delete for products and suppliers, placeOrder(), displayOrders(), report(), saveAll(), loadAll().
FileManager	Reads and writes data files.	Functions: saveLines(), loadLines(), split().

main.cpp only creates the Warehouse object, shows the menu and calls Warehouse functions. All business logic is kept inside the classes.

7. OOP Concepts Used

- **Classes and objects:** Each real-world entity (product, supplier, stock, order) is a class. For example, Product p(1, "Laptop", "Electronics", 55000); creates an object.
- **Encapsulation:** Data members such as price, phone and quantity are private or protected and are changed only through member functions. setPrice() rejects negative values, so an object can never hold invalid data.
- **Constructors and destructors:** Every class has a default constructor (for example Product()) and a parameterized constructor (for example Product(id, name, category, price)). Destructors are defined in all classes; the Warehouse destructor releases all dynamic memory, and base classes have virtual destructors.
- **Inheritance:** Product and Supplier inherit from Item, reusing id and name. IncomingOrder and OutgoingOrder inherit from Order, reusing orderId, productId, quantity and date.
- **Runtime polymorphism:** Order declares a pure virtual function process(Stock&). Warehouse stores orders as Order* pointers and calls o->process(stock). At run time, IncomingOrder adds stock while OutgoingOrder removes stock, with no if-else on the order type inside process. Item likewise declares virtual display() and toFileString().
- **Abstract classes and virtual functions:** Item and Order cannot be instantiated; they force derived classes to provide their own version of display(), toFileString() and process().
- **Validation and exception handling:** Invalid input throws exceptions (invalid_argument, runtime_error) such as "Price cannot be negative", "Phone must be 10 digits",

"Insufficient stock", "Product ID already exists". The menu loop in main.cpp catches them with try/catch, prints the message and continues, so the program never crashes.

- **Modular design:** Each class is split into a .h and a .cpp file, 14 source files in total.

Dynamic memory

The Warehouse class stores its data in arrays created with new[] instead of fixed-size arrays: Product* products = new Product[productCap]; and similarly for suppliers, stocks and Order** orders. The arrays start with a capacity of 5. When an array becomes full, a new array of double size is created with new[], the old elements are copied, and the old array is released with delete[]. This lets the warehouse hold any number of records without wasting memory in advance. Each order object is created individually with new IncomingOrder(...) or new OutgoingOrder(...). In the Warehouse destructor every order is released with delete orders[i], and then delete[] is used for all four arrays, so there are no memory leaks. If an order fails (for example insufficient stock), the order object is deleted immediately before the exception is passed on.

8. File Handling

All data is stored in plain text files in the Data folder. Each record is one line and fields are separated by the | character.

File	Contents	Example line
products.txt	id name category price	1 Laptop Electronics 60000.000000
suppliers.txt	id name phone city	1 Ravi Traders 9876543210 Rajkot
stocks.txt	productId quantity reorderLevel	1 30 5
orders.txt	orderId type productId quantity date	101 INCOMING 1 20 03/10/2026

Saving

saveAll() converts every object to a text line using toFileString() and passes the lines to FileManager::saveLines(), which writes them with an ofstream. Saving happens when the user chooses Save, and automatically when the user chooses Exit.

Loading

At program start, loadAll() reads each file with FileManager::loadLines() (an ifstream), splits each line with FileManager::split(), rebuilds the objects and stores them in the dynamic arrays. If a file does not exist (first run), an empty list is returned and the program starts empty. While loading orders, stock is not processed again because the saved stock quantity already includes those orders.

Persistence test

A product, a supplier and two orders were entered and the program was exited with option 9. The program was then started again; it printed "Data loaded" and the same records and stock quantity were displayed, confirming persistence (Figure 14).

9. UML Diagrams

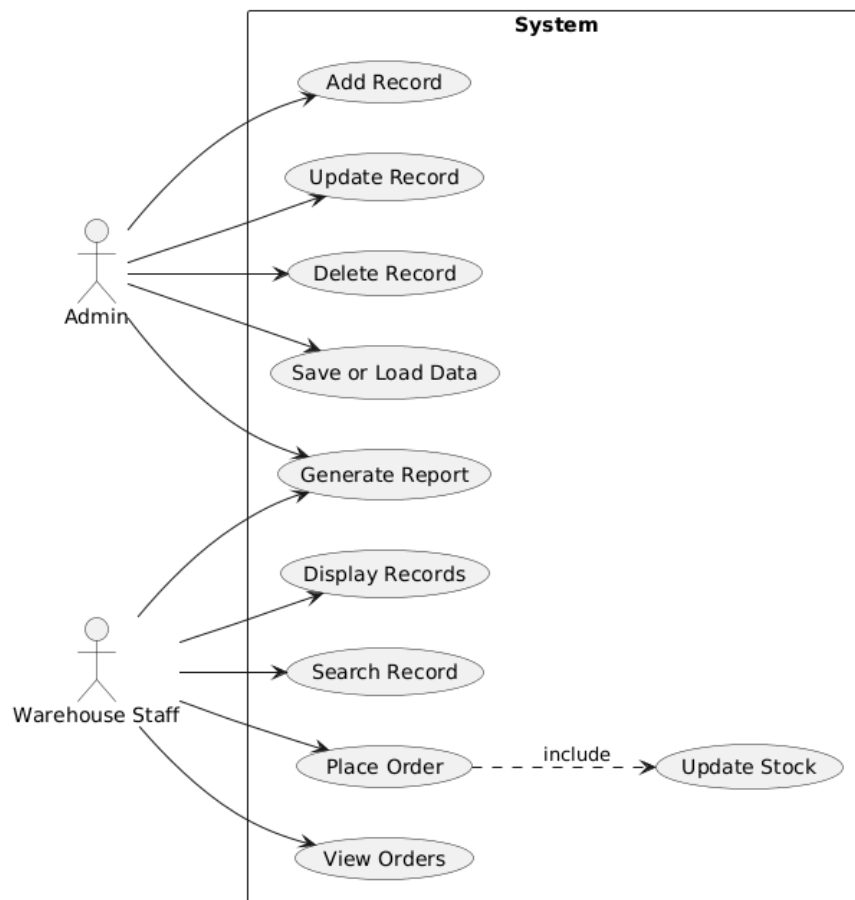


Figure 1: Use Case Diagram

The Use Case Diagram has two actors, Admin and Warehouse Staff, and ten use cases. Placing an order includes the Update Stock use case.

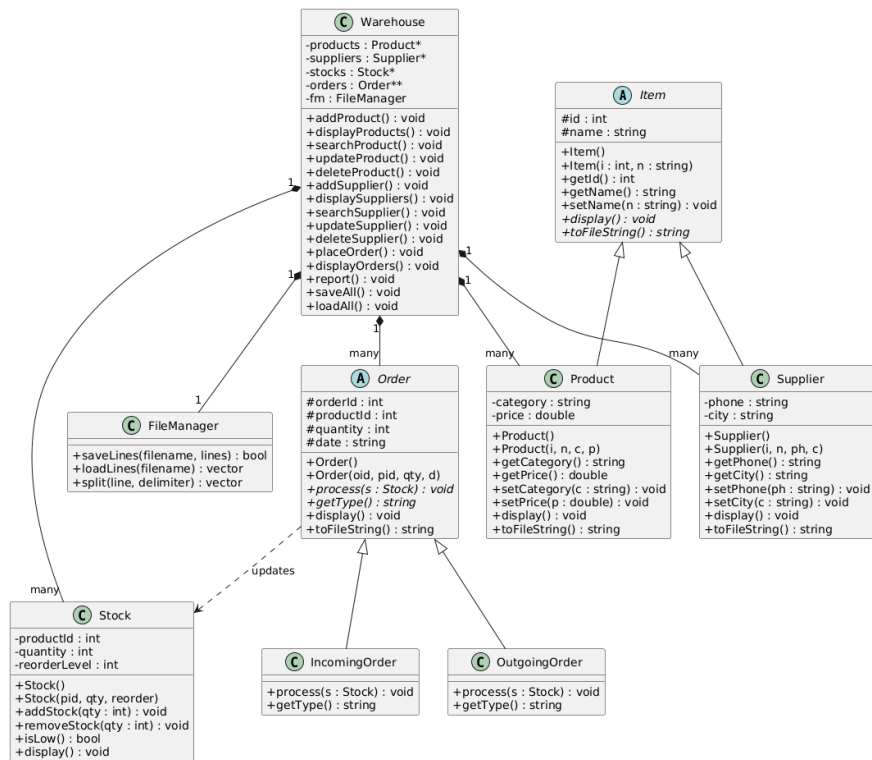


Figure 2: Class Diagram

The Class Diagram shows the inheritance of Product and Supplier from Item, and of IncomingOrder and OutgoingOrder from Order. Warehouse owns (composition) the products, suppliers, stocks, orders and the FileManager.

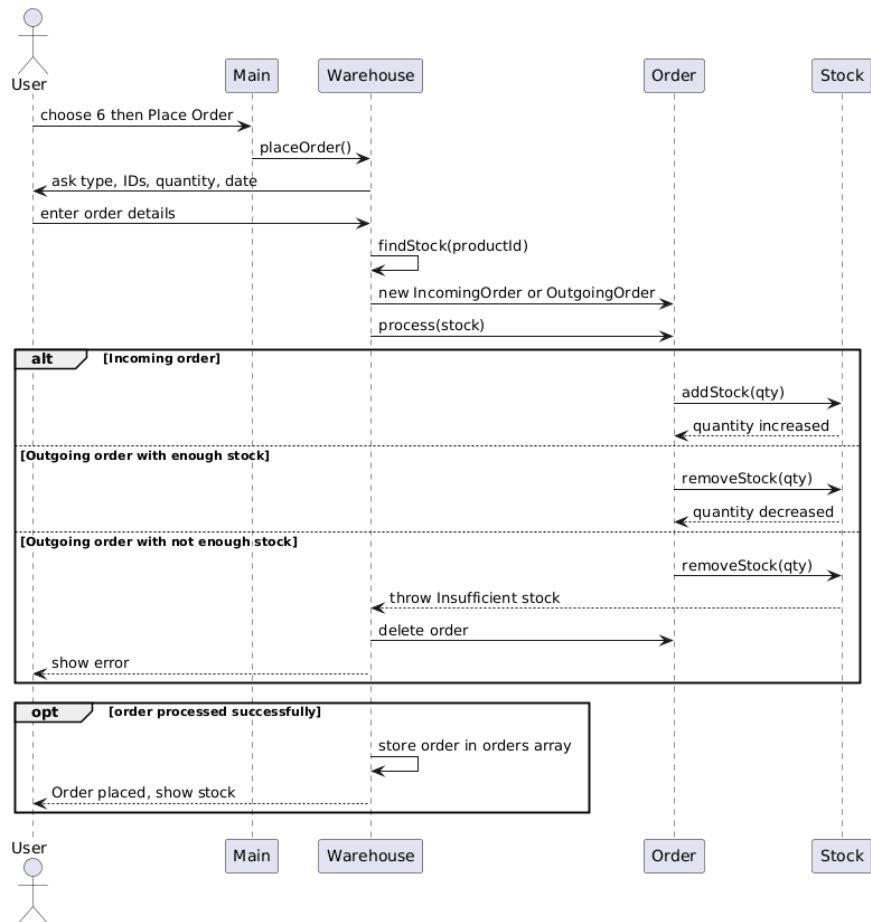


Figure 3: Sequence Diagram 1 - Place Order (main transaction)

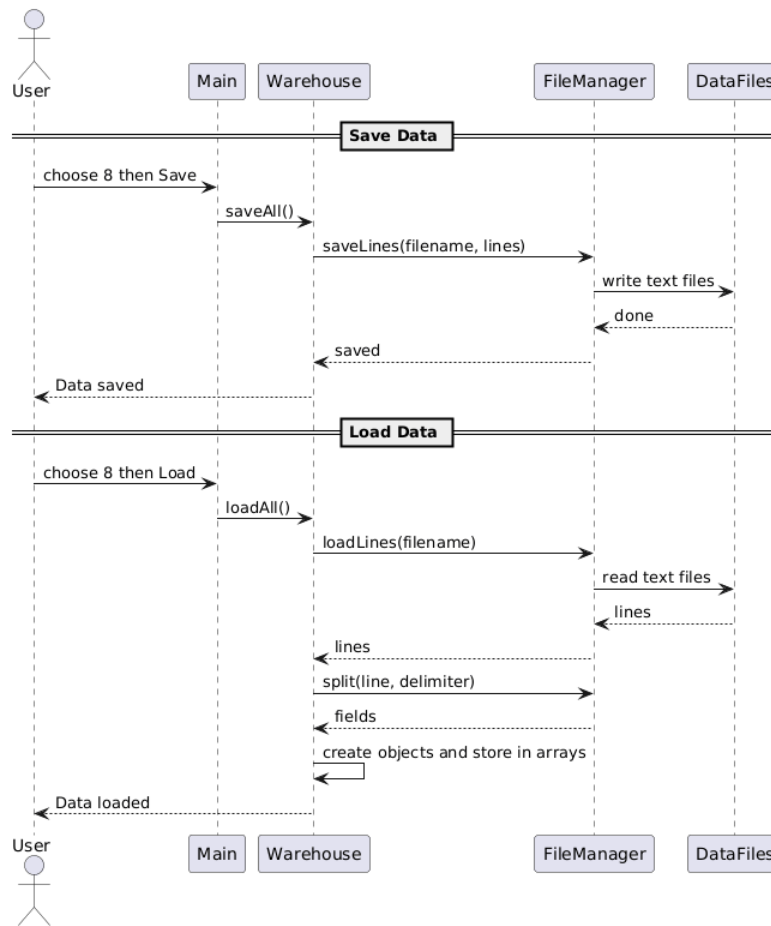


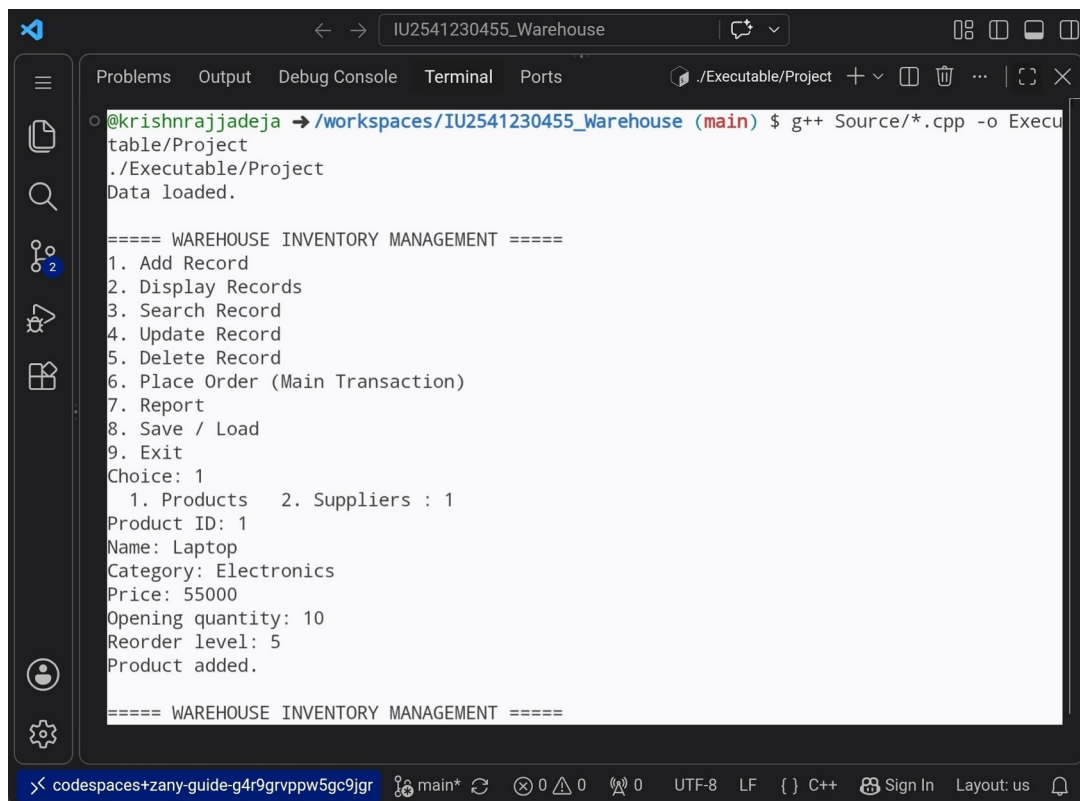
Figure 4: Sequence Diagram 2 - Save and Load Data

10. Program Workflow

1. The program starts and main() creates a Warehouse object. The constructor allocates the dynamic arrays.
2. loadAll() reads the data files and restores earlier records (if they exist).
3. The main menu is displayed and the user selects an option from 1 to 9.
4. main() calls the matching Warehouse function. Warehouse works with Product, Supplier, Stock and Order objects.
5. For a transaction (option 6), Warehouse creates an IncomingOrder or OutgoingOrder and calls process(); the order updates the Stock object. If the quantity is not available, an exception is thrown and displayed.
6. Any invalid input throws an exception which is caught in the menu loop; the program shows the error and returns to the menu.
7. On Exit (option 9), saveAll() writes all data through FileManager to the data files.
8. The Warehouse destructor releases all dynamic memory and the program ends.

11. Screenshots

The following screenshots show the important operations and successful execution of the program.

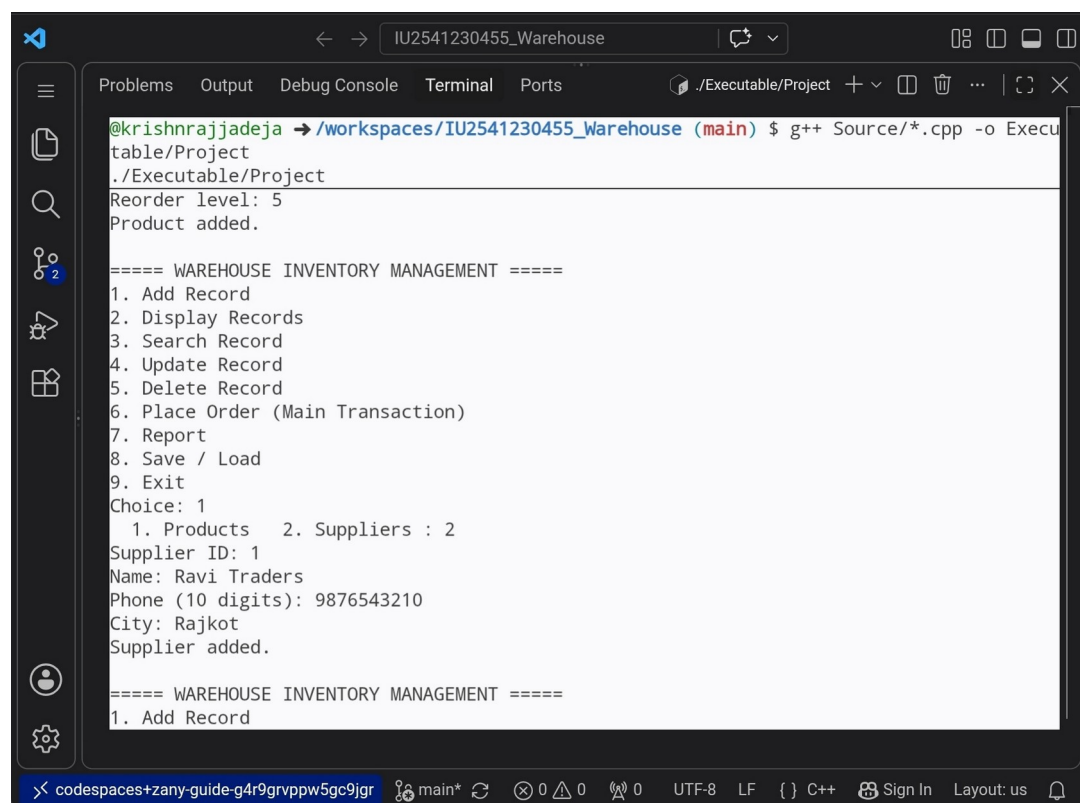


```
@krishnrajjadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
Data loaded.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 1
    1. Products    2. Suppliers : 1
Product ID: 1
Name: Laptop
Category: Electronics
Price: 55000
Opening quantity: 10
Reorder level: 5
Product added.

===== WAREHOUSE INVENTORY MANAGEMENT =====
```

Figure 5: Adding a product (Option 1 - Products)



```
@krishnrajjadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
Reorder level: 5
Product added.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 1
    1. Products    2. Suppliers : 2
Supplier ID: 1
Name: Ravi Traders
Phone (10 digits): 9876543210
City: Rajkot
Supplier added.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
```

Figure 6: Adding a supplier (Option 1 - Suppliers)

```

@krishnrajadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
1 | Laptop | Electronics | Rs.55000
  Product 1 | Qty: 10 | Reorder at: 5

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 4
  1. Products  2. Suppliers : 1
Enter Product ID to update: 1
New name: Laptop
New category: Electronics
New price: 60000
Product updated.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record

```

Figure 7: Displaying a product with its stock, then updating it (Options 2 and 4)

```

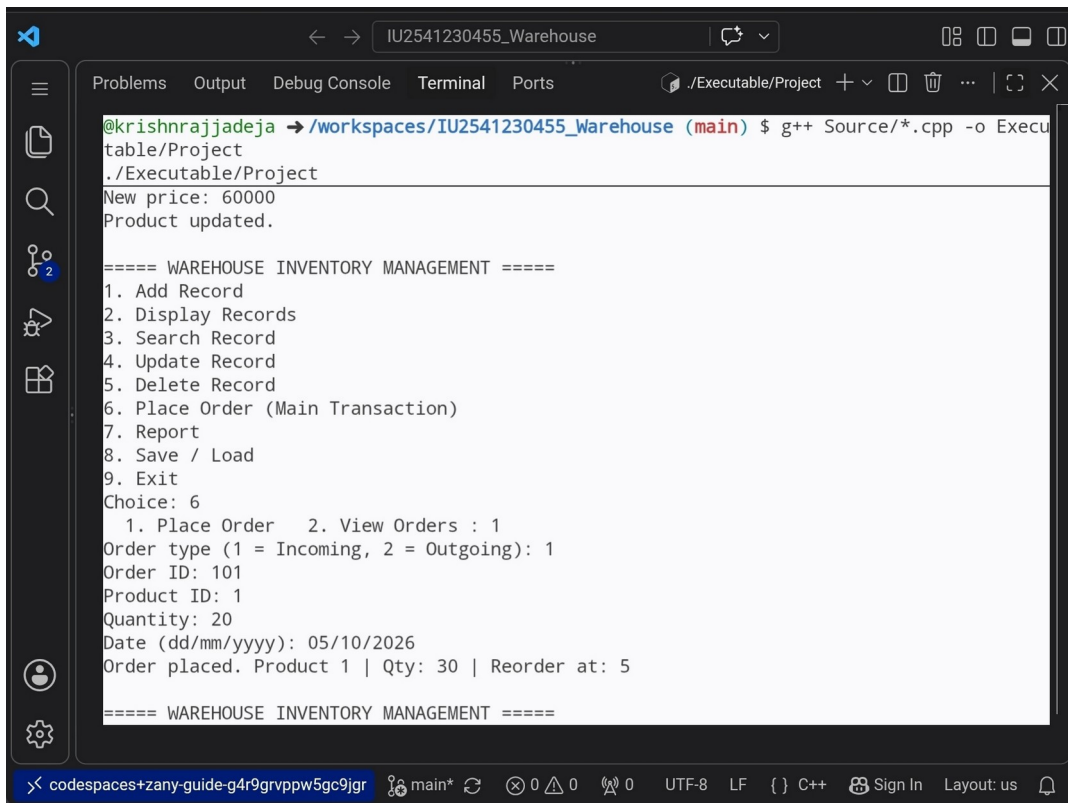
@krishnrajadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
Phone (10 digits): 9876543210
City: Rajkot
Supplier added.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 3
  1. Products  2. Suppliers : 1
Enter Product ID: 1
1 | Laptop | Electronics | Rs.55000
  Product 1 | Qty: 10 | Reorder at: 5

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records

```

Figure 8: Searching a product by ID (Option 3)

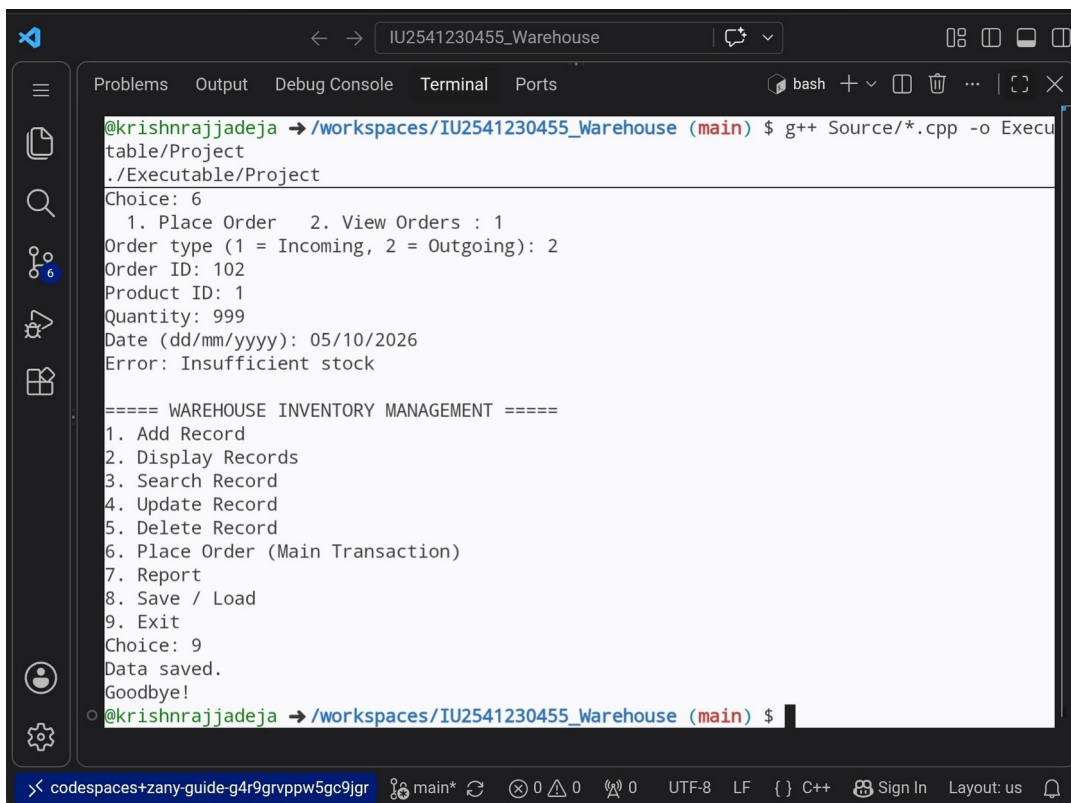


```
@krishnradjadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
New price: 60000
Product updated.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 6
    1. Place Order    2. View Orders : 1
Order type (1 = Incoming, 2 = Outgoing): 1
Order ID: 101
Product ID: 1
Quantity: 20
Date (dd/mm/yyyy): 05/10/2026
Order placed. Product 1 | Qty: 30 | Reorder at: 5

===== WAREHOUSE INVENTORY MANAGEMENT =====
```

Figure 9: Incoming order - stock increases to 30 (Option 6)



```
@krishnradjadeja → /workspaces/IU2541230455_Warehouse (main) $ g++ Source/*.cpp -o Executable/Project
./Executable/Project
Choice: 6
    1. Place Order    2. View Orders : 1
Order type (1 = Incoming, 2 = Outgoing): 2
Order ID: 102
Product ID: 1
Quantity: 999
Date (dd/mm/yyyy): 05/10/2026
Error: Insufficient stock

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 9
Data saved.
Goodbye!
@krishnradjadeja → /workspaces/IU2541230455_Warehouse (main) $
```

Figure 10: Outgoing order rejected - Insufficient stock, then Exit with auto-save

```

@krishnrajadeja → /workspaces/IU2541230455_Warehouse (main) $ ./Executable/Project
Error: Insufficient stock

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 7
===== WAREHOUSE REPORT =====
Products : 1
Suppliers: 0
Orders   : 1
Total stock value: Rs.1.65e+06
--- Low stock items ---
None.

```

Figure 11: Warehouse report (Option 7)

```

@krishnrajadeja → /workspaces/IU2541230455_Warehouse (main) $ ./Executable/Project
9. Exit
Choice: 1
1. Products 2. Suppliers : 1
Product ID: 2
Name: Pen
Category: Stationery
Price: 10
Opening quantity: 5
Reorder level: 2
Product added.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 5
1. Products 2. Suppliers : 1
Enter Product ID to delete: 2
Product deleted.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records

```

Figure 12: Adding a spare product and deleting it (Option 5)

```

@krishnrajadeja → /workspaces/IU254123045
5_Warehouse (main) $ ./Executable/Project
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 2
  1. Products  2. Suppliers : 1
  1 | Laptop | Electronics | Rs.60000
    Product 1 | Qty: 30 | Reorder at: 5

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 9
Data saved.
Goodbye!
@krishnrajadeja → /workspaces/IU254123045_Warehouse
o (main) $

```

Figure 13: Display after delete - only the Laptop remains; Exit auto-saves

```

@krishnrajadeja → /workspaces/IU254123045_Warehouse (main) $ g++ Source/*.cpp -o Execu
table/Project
./Executable/Project
Choice: 9
Data saved.
Goodbye!
@krishnrajadeja → /workspaces/IU254123045_Warehouse (main) $ ./Executable/Project
Data loaded.

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record
2. Display Records
3. Search Record
4. Update Record
5. Delete Record
6. Place Order (Main Transaction)
7. Report
8. Save / Load
9. Exit
Choice: 2
  1. Products  2. Suppliers : 1
  1 | Laptop | Electronics | Rs.60000
    Product 1 | Qty: 30 | Reorder at: 5

===== WAREHOUSE INVENTORY MANAGEMENT =====
1. Add Record

```

Figure 14: Persistence - data loaded after restarting the program

12. Testing

No .	Test case	Input	Expected output	Actual result
1	Add product	ID 1, Laptop, Electronics, 55000, qty 10, reorder 5	Product added.	Pass
2	Add supplier	ID 1, Ravi Traders, 9876543210, Rajkot	Supplier added.	Pass
3	Display records	Option 2, products then suppliers	Records and stock shown	Pass
4	Search product	Option 3, ID 1	Laptop details shown	Pass
5	Update product	Option 4, ID 1, new price 60000	Product updated.	Pass
6	Incoming order	Type 1, order 101, product 1, qty 20	Order placed, Qty 30	Pass
7	Outgoing order, not enough stock	Type 2, order 102, product 1, qty 999	Error: Insufficient stock	Pass
8	Duplicate ID	Add product with existing ID 1	Error: Product ID already exists	Pass
9	Delete product	Option 5, Products, ID 2	Product deleted.	Pass
10	Report	Option 7	Counts, total value, low stock list	Pass
11	Exit and save	Option 9	Data saved. Goodbye!	Pass
12	Persistence	Restart program, option 2	Data loaded; saved records shown	Pass

13. Challenges and Solutions

- **Keeping stock correct on load:** When orders were reloaded from file, processing them again would have changed the stock a second time. The solution was to create the order objects on load without calling process(), because stocks.txt already holds the final quantity.
- **Memory leak on failed orders:** An order object is created before it is processed; if processing failed, the object was never deleted. This was solved by catching the exception, deleting the object and re-throwing the exception.
- **Fixed array size:** Fixed arrays limited the number of records. Dynamic arrays that double in size when full (new[] / delete[]) solved this.

- **Mixed input with cin and getline:** Reading a number with cin left a newline in the buffer which broke the next getline. This was solved with cin.ignore() before reading text, and by clearing the input stream after errors.
- **Tools and diagrams:** The work was done on a tablet using GitHub Codespaces. Some UML diagrams initially rendered without text labels; they were fixed by rewriting the PlantUML code in a simpler form. A temporary network timeout during git push was solved by pushing again.

14. Conclusion

The Warehouse Inventory Management System successfully manages products, suppliers, stock and orders through a menu-driven C++ program. The project applied classes and objects, encapsulation, constructors and destructors, inheritance, runtime polymorphism, dynamic memory, file handling, validation and exception handling in a modular .h/.cpp structure. Data persists between runs, and the UML diagrams document the design. Through this project I learned how to divide a problem into classes, how virtual functions let each order type behave differently through a common base pointer, and how to manage dynamic memory safely. As future improvements, the system could use a database, add user login, and support searching by name or category.

GitHub Repository: https://github.com/krishnrajjadeja/IU2541230455_Warehouse